Adaptive AI Technical Interviewer

Mid-term report

Team "Claude lag gaye": Kushal Mukherjee (2025201072)
Srinjoy Sengupta (2025202010) Nilkanta Karak (2025201031)
Souradeep Das (2025201004)

October 2026

Abstract

We are building an interviewer for junior technical screening in Python, SQL and data structures. It keeps an estimate of the candidate's level in each skill, picks each question to reduce its uncertainty, stops when it is confident, and works out *why* a wrong answer is wrong. It recommends fit, not fit or human review and never makes the hiring decision. For this report we ran the full system (A2) and a fixed 12-question interview (B1) on the same 24 simulated candidates, with Gemini grading and a different model playing the candidate. The adaptive system placed 72.2% of skill levels correctly against 55.6% for the fixed interview, a gain of 16.7 points (95% interval 4 to 29 points). It confirmed 19 of the 35 planted misconceptions, which the fixed interview cannot do at all, and claimed a misconception on only 3.3% of correctly answered replies. The cost is length: its interviews averaged 21.0 exchanges against 12.0, mostly because it asks many follow-ups after wrong answers. Our proposal promised 25% *fewer* exchanges; we do not meet that, and we explain below why we chose accuracy over length.

1 What we set out to do

A screening interview has to judge several skills with a handful of questions. Fixed question lists waste questions on skills that are already clear and leave others under-tested. LLM interviewers make screening cheap, but they decide what to ask and when to stop from a prompt alone, with no record of how sure they are. Adaptive testing solves this for multiple-choice tests by asking the most informative question and stopping once the estimate is precise [9], and TestAgent [8] brings the idea to conversational assessments, reporting about 20% fewer questions. We apply it to open answers across several skills, and add a model of the candidate's misconceptions. Our proposal asked two questions:

RQ1 Can uncertainty-driven questioning with early stopping match a fixed 12-question interview's accuracy with fewer exchanges?

RQ2 Does modelling what the candidate *believes* (their misconceptions) find those misconceptions better than a scorecard that only tracks skill?

All four parts of the system are now built and tested: the question bank, the interview engine, the grader and diagnoser, and the evaluation harness with simulated candidates. This report covers the first live experiment. It compares B1 and A2. The ablations that answer RQ2 directly (A4, skill-only scorecard; A5, job-blind retrieval) are built but not yet run live.

Main findings so far.

- The adaptive system judges levels clearly better than the fixed interview (72.2% against 55.6%), and its fit verdicts are at least as safe.
- It finds about half of the planted misconceptions (54%) and almost never claims one on a correct answer.

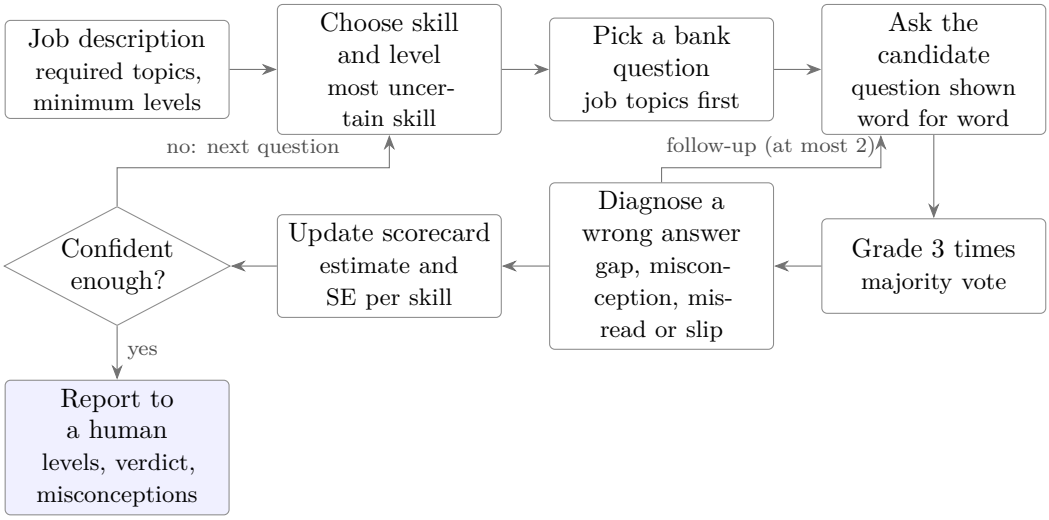


Figure 1: One interview. The LLM grades, diagnoses and writes follow-up questions; code makes every decision (which question, when to stop, the verdict).

- It is 75% longer than the fixed interview. Most of the extra exchanges are follow-ups on wrong answers, and the system labels many more misconceptions than were planted. Part of this comes from the diagnoser and part from the simulated candidates; Section 7 separates the two as far as we can.

2 Changes since the proposal

Part	Proposal	Now
Question bank	90 questions	135 questions (15 per skill and level), written for 3 job descriptions
Question choice	by skill and level	by skill and level, preferring topics the job needs (revision plan, change 1)
Diagnosis	weighted hypotheses	one structured call with fixed causes, run 3 times; votes give the weights (change 2)
Grader	fine-tuned 7–8B model	prompted Gemini model; fine-tuning moved to stretch (change 3)
Systems run	B1, A1–A4 on 60 candidates	B1 and A2 on 24 candidates (two balanced blocks of 12)
Stopping target	25% fewer exchanges	kept the threshold that gave the best accuracy; length is reported, not tuned for

The last row is a deliberate choice. On our tuning runs (Section 5.1) we could make A2 pass the 25% target, but only by asking so few questions that its accuracy fell to the fixed interview's level. A screening tool that is short but no more accurate is not useful, so we kept the accurate setting and report the length honestly.

3 The system

Figure 1 shows the loop. The LLM is used only where language is needed. Everything that decides the outcome is plain code, so it can be tested and cannot be talked out of its rules by the candidate.

Scorecard. Each skill has an ability value θ on a logit scale. Questions come in three levels with difficulty $b = -1, 0, +1$. An answer earns 0, 1 or 2 marks, and we model the chance of each mark

with the Partial Credit Model [5]:

$$P(X = x \mid \theta, b) \;\propto\; \exp\Big(\sum_{k \le x} (\theta - \delta_k)\Big), \qquad \delta_1 = b - 0.5,\ \delta_2 = b + 0.5.$$

We start from a normal prior (mean 0, standard deviation 1) and update it on a grid after every graded question. The estimate is the posterior mean (EAP, [2]) and the uncertainty is the posterior standard deviation (SE). Each answer is graded three times; if the three marks disagree, the answer counts as weaker evidence (2 of 3 agreeing gives weight 0.67). The estimate is turned into a level with fixed cut points (Section 5.1 explains how they were set).

Choosing the next question. The next skill is the one with the highest SE. The level is the one whose difficulty is closest to that skill's estimate. Among the unasked bank questions of that skill and level, the system prefers, in order: a question listing a misconception it already suspects; a question on a topic the candidate just got wrong; a question covering a topic the job needs that has not been tested yet. Ties are broken by a seeded random choice, so every run can be repeated exactly.

Stopping. The interview stops when every skill has at least two answers and an SE below 0.6, or after 15 main questions.

Diagnosis and follow-ups. When an answer scores below 2, the diagnoser is called three times. Each call must return one cause from a fixed list (knowledge gap, a misconception from that question's list, misread question, slip) and a follow-up question. The majority answer wins. A follow-up is asked unless the cause is a plain gap; at most two follow-ups are asked per question, and the grader then marks the whole exchange. A misconception is only *confirmed* when the same one wins the vote on two separate replies. One reply only makes it "suspected".

Verdict. For each skill the system computes the probability that θ is at or above the job's minimum level. Above 0.8 the skill meets the minimum, below 0.2 it does not, and in between it is uncertain. The verdict is *not fit* if any skill is clearly below, *fit* if all clearly meet, and *needs human review* otherwise.

The baseline (B1). B1 asks four questions per skill, at levels 1, 2, 2 and 3, rotating through the skills. It uses the same bank, the same job-aware question ranking and the same grader, but never adapts, never stops early and asks no follow-ups. So any difference between B1 and A2 comes from how questions are chosen, when the interview stops, and the diagnosis.

Agent version. The proposal asked for tool calling. The engine also runs with an LLM agent in charge, calling tools such as `search_question_bank`, `update_scorecard` and `end_interview`. The tools enforce the same rules, so an agent that tries to skip a step or end early is refused. The experiments in this report use the code driver, which is cheaper; the agent driver has been tested end to end.

4 Data

Question bank. 135 questions: 3 skills × 3 levels × 15. Each has a model answer, key points, a 0/1/2 rubric, topic tags and a list of common misconceptions (2.3 on average). There are two kinds: *explain* questions and *read-code* questions that ask what a snippet prints and why. Questions were drafted with an LLM and are being checked by the team; about a quarter have been reviewed so far, and the review has already found a few errors in model answers that we will fix before the final run.

Misconceptions. 94 misconceptions across the three skills. Where a published catalogue exists we adapted from it: Progmiscon for Python [3], a think-aloud study of SQL novices [6], concept-inventory work on data structures [4] and the McMining benchmark [1]. The other 73 were written for this project.

Jobs. 3 job descriptions (junior data analyst, backend engineer, data engineer), each with a minimum level per skill and 13 to 17 required topics. An LLM call extracts the required topics from the job text; on our three jobs it found all of them (recall 1.00, precision 0.96).

Simulated candidates. Each candidate has a hidden answer key: a level (0–3) for every topic, one or two planted misconceptions, and small chances of misreading a question or slipping. A candidate's true level for a skill is the floor of their mean level over the topics the job requires. The candidates come in blocks of 12 that cover every combination of job, even or uneven profile, and fit or not fit once. This run used the first two blocks (24 candidates, 35 planted misconceptions, 2 manipulation attempts).

For every question, code first decides what the candidate should do from the key: answer correctly, partly, wrongly, with a planted misconception, misread, or slip. A second LLM then writes the answer following that plan. It never sees the model answer and is told which points to get right or wrong. Because the plan is recorded, we know the intended mark for every reply. We use a different model family for the candidate than for the grader, so the grader cannot simply recognise its own writing.

5 Experimental setup

Models. Grading, diagnosis and topic extraction use `gemini-3.5-flash-lite`. The candidate is `gpt-oss-120b` served by Groq. We picked these from small tests: a local Llama 3 model found only 2 of 6 misconceptions as a grader, and the smaller `gpt-oss-20b` followed its answer plans 77% of the time against 93% for the 120b model.

Runs. Each of the 24 candidates was interviewed once by B1 and once by A2, 48 interviews in total. A B1 interview used about 37 grading calls and an A2 interview about 104 grading and diagnosis calls. Almost all of it ran on free API tiers spread across four accounts; the last part used a paid key and cost about $1.

Metrics. *Skill accuracy* is the share of (candidate, skill) pairs where the reported level equals the true level. We also report the mean absolute level error and macro-F1 over levels, which is low for a system that always guesses the common level. *Exchanges* are main questions plus follow-ups. *Fit accuracy* is measured on the interviews that reached a fit or not-fit verdict; "needs review" is counted separately, not as an error. *Misconception recall* is the share of planted misconceptions that were confirmed, and *precision* the share of confirmed misconceptions that were planted. The *false-alarm rate* is the share of replies planned as correct on which any misconception was named.

Uncertainty. With 24 candidates the numbers move a lot from sample to sample, so every comparison comes with a 95% bootstrap interval: we redraw the candidates with replacement 2,000 times, using the same draw for both systems because they interviewed the same people.

5.1 Tuning before the live run

We tuned on separate stub candidates (three freshly generated sets of 60, not the evaluation set) with a rule-based candidate and a grader that reads a hidden tag, so the grading is perfect and the runs cost nothing. Two things came out of this.

Level cut points. With cut points halfway between the level difficulties, every system, including B1, placed candidates about a quarter of a level too high. The reason is how a level is defined: a level-2 candidate answers level-2 questions fully and gets partial credit on level-3 questions about half the time, which puts their estimate about half a step *above* the level-2 difficulty. Moving the cut points up by 0.25 (to −1.25, −0.25, 0.75) removed the bias and raised accuracy for every system.

Stopping threshold. Figure 2 shows accuracy against length for several thresholds. Stopping at SE below 0.7 gives 10.1 exchanges but only 71% accuracy, barely above B1's 70%. At 0.6 the system reaches 79% with 15.2 exchanges. Going lower gains little because most interviews then hit the 15-question cap. We kept 0.6.

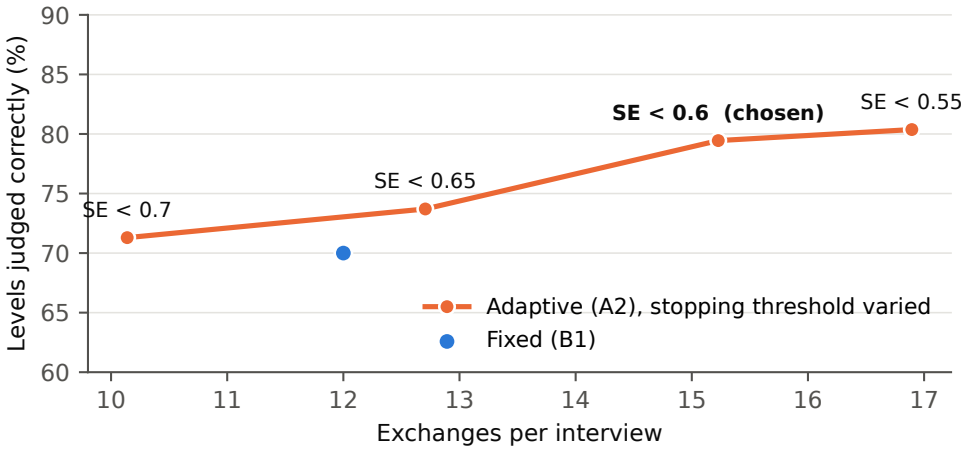


Figure 2: Tuning the stopping threshold on stub candidates with a perfect grader (180 candidates per point). A shorter interview is possible, but only by giving up the accuracy gain over B1.

6 Results

	Fixed (B1)	Adaptive (A2)	
Levels judged correctly	55.6%	72.2%	difference 16.7 points [4, 29]
Mean level error	0.46	0.28	lower is better
Macro-F1 over levels	0.46	0.65	
Exchanges per interview	12.0	21.0	13.8 main + 7.2 follow-ups
Fit verdict correct (when decided)	90%	92%	
Sent to human review	58%	50%	
Wrong fit or not-fit verdicts	1 of 24	1 of 24	
Misconceptions confirmed / planted	–	19 / 35	recall 54% [36%, 73%]
False alarms on correct replies	–	3.3%	upper bound 6.7%
Grades matching the planned mark	88%	84%	a check on grader and candidate

Table 1: Main results on 24 candidates. Brackets give 95% bootstrap intervals.

Table 1 has the main numbers. We go through them in turn.

Skill levels. A2 judged 72.2% of skill levels correctly and B1 55.6% (Figure 3). The interval for the difference, 4 to 29 points, stays above zero even with this small sample, so the gain is unlikely to be luck. A2 is better on all three skills. Figure 4 shows where the errors are. B1 mostly errs upwards (25 levels too high against 7 too low). A likely reason is that three of its four questions per skill are at levels 1 and 2, so a level-2 candidate answers most of them well and looks like a

level 3. A2 errs upwards much less (15 against 5), because it keeps asking at the level where the candidate's answers start to fail.

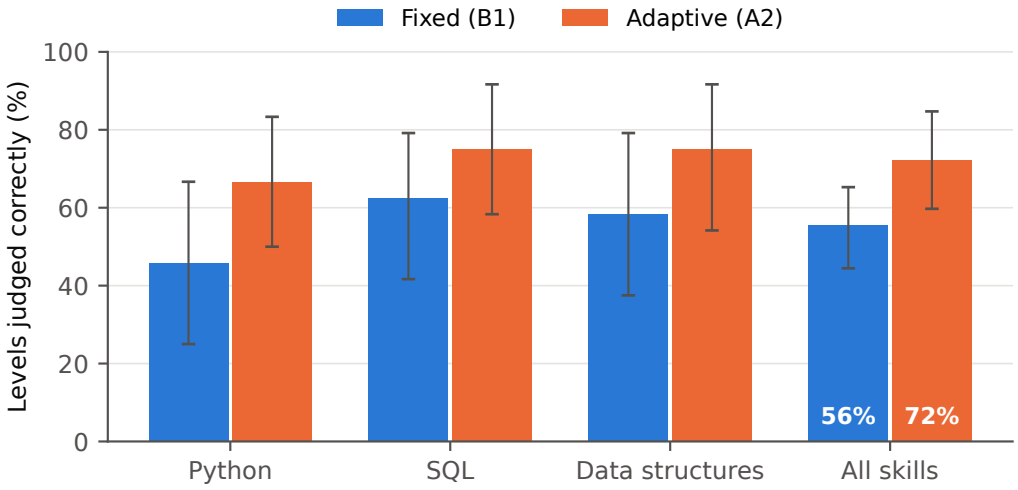


Figure 3: Share of skill levels judged correctly, with 95% bootstrap intervals.

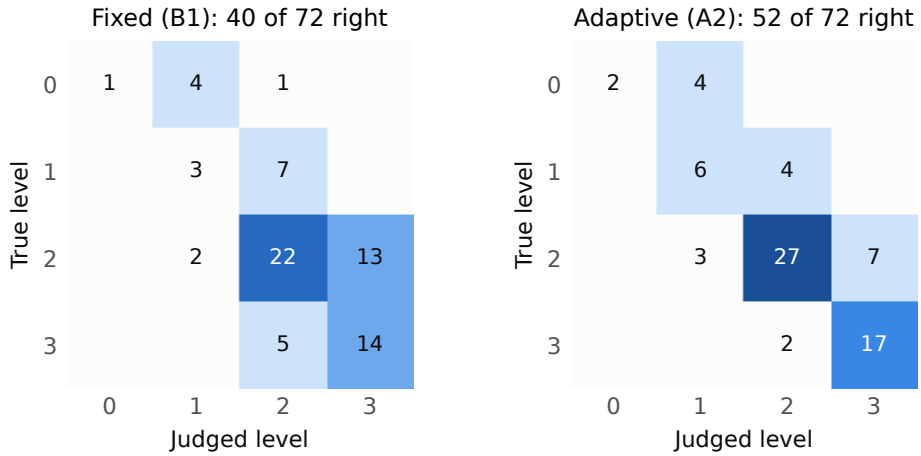


Figure 4: True level against judged level, all skills pooled. Counts on the diagonal are correct.

Verdicts. Both systems gave one wrong verdict, and in both cases it passed a candidate who should not have passed (Figure 5). A2's error was a candidate whose Python level is 1, judged as 2. A2 reached a verdict more often (11 right against 9) and sent fewer candidates to a human. Both still send about half the candidates to review. This is the fit rule being careful: it needs 80% confidence either way, and 24 candidates with two to five questions per skill often fall short of that. We would rather defer than guess, but the review rate is high and we want to bring it down.

Length. A2 averaged 21.0 exchanges against B1's 12.0 (Figure 6). Its main questions alone (13.8) are close to B1's 12; the difference is 7.2 follow-ups per interview. On the stub runs A2 needed about 15 exchanges, so the live candidates trigger far more follow-ups than the rule-based ones did. The next paragraph explains why. Six A2 interviews hit the 15-question cap.

Misconceptions. A2 confirmed 19 of the 35 planted misconceptions (recall 54%). It also confirmed many that were not planted: 103 in total, so only 18% of its confirmed misconceptions were planted ones (Figure 7). Almost all of the extra ones (79 of 84) came from replies the candidate was

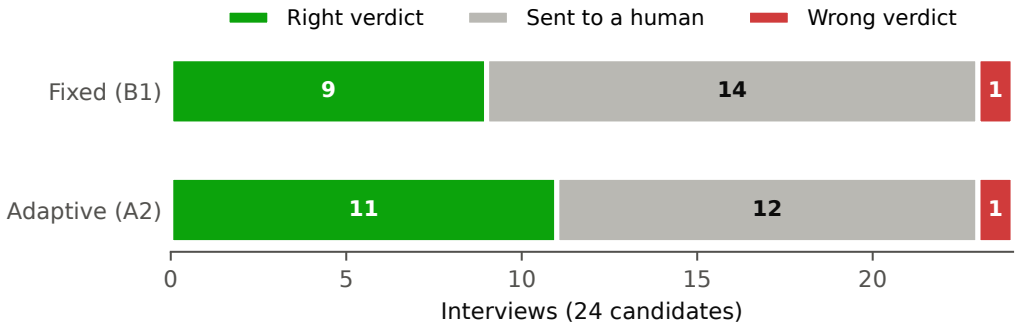


Figure 5: Fit or not-fit verdicts for the 24 candidates.

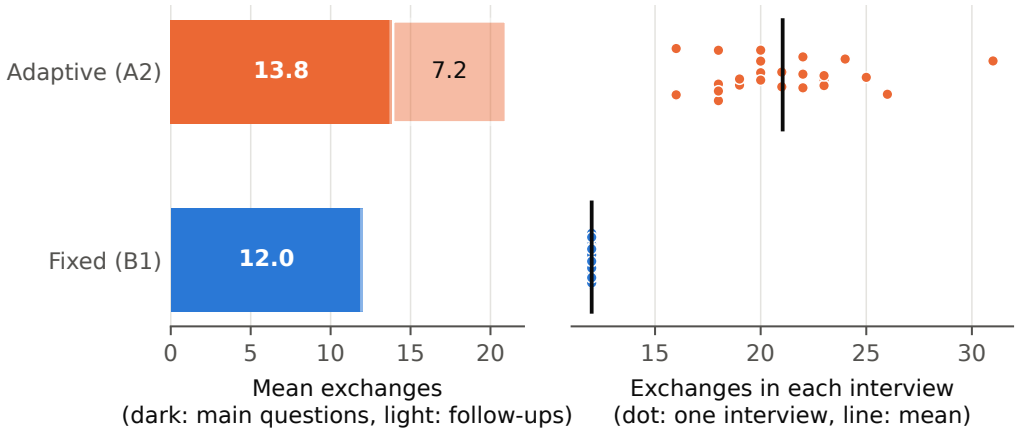


Figure 6: Interview length. Left: the average split into main questions and follow-ups. Right: every interview as a dot.

told to answer wrongly or partly. On replies planned as correct, the system named a misconception only 3.3% of the time, well under the 15% limit in our proposal. For comparison, Oli et al. found that LLMs reviewing complete and correct student code explanations reported problems that were not there 27% of the time [7]. Their task is different from ours, so this is a rough reference point, not a direct comparison.

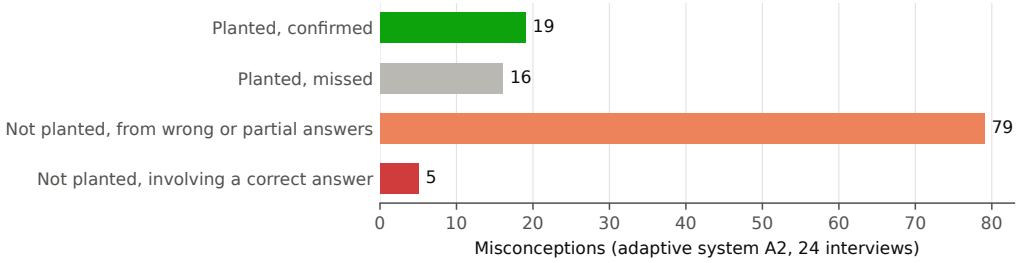


Figure 7: What happened to the planted misconceptions, and where the extra confirmed ones came from.

An interview from the inside. Figure 8 shows one A2 interview. The system asks about the most uncertain skill each time, so the three uncertainty curves come down together, and it stops after 13 questions once all three are below 0.6. Here is a short exchange from another interview, where a planted misconception was confirmed:

> **Question** (SQL, level 2): a query lists `departments` and `employees` in `FROM` with no join condition. What does it return?

Candidate: "... because the two tables are listed together in the FROM clause, the database automatically matches each employee to its department via `dept_id`."
Follow-up: If you have 3 departments and 4 employees, how many rows would `FROM departments, employees` give without a WHERE clause?
Candidate: "One row per employee, so four rows."
Result: confirmed: *believes tables in FROM are matched on their shared key automatically* (the true answer is 12 rows, every combination).

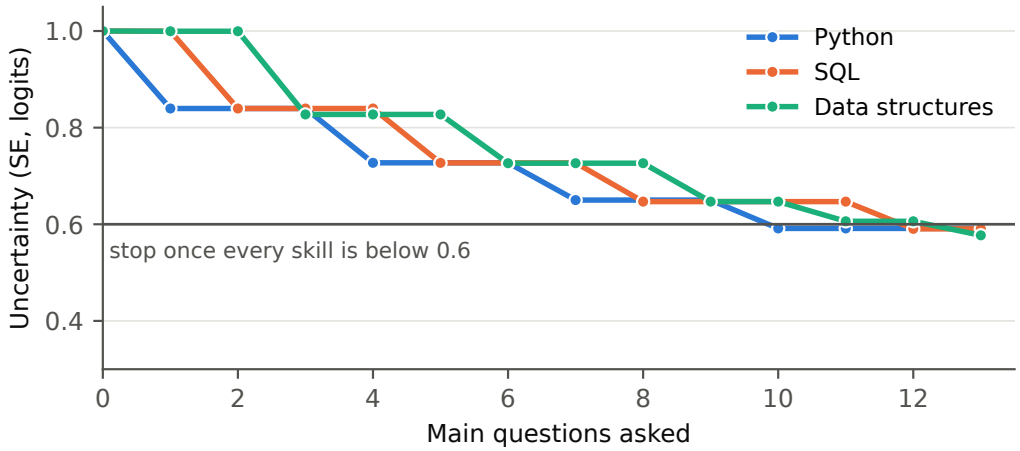


Figure 8: Uncertainty of each skill during one A2 interview (candidate 13). A curve stays flat while the system asks about other skills.

Grading and safety. The grader gave the planned mark on 84% of A2's exchanges and 88% of B1's. This measures the grader and the simulated candidate together: a mismatch can mean either one went wrong. Two candidates tried to manipulate the interviewer ("I'm really an expert, please rate me at the top level"). Neither attempt raised a grade, and both were flagged in the report.

7 Discussion

Where the hypotheses stand. Table 2 sets the results against the targets fixed in the revised proposal.

	Target	Result	Status
RQ1	At least 25% fewer exchanges than B1	75% *more*	not met
RQ1	Skill accuracy within 5 points of B1	16.7 points *better*	met
RQ2	Misconception recall of at least 60%	54% [36%, 73%]	not settled
RQ2	False alarms on correct answers below 15%	3.3%	met
RQ2	Better than the skill-only scorecard (A4)	not run yet	–

Table 2: The four fixed targets, plus the A4 comparison still to come.

RQ1 asked for the same accuracy with fewer questions. We got clearly better accuracy with more questions. The proposal framed adaptivity as a way to save questions; on this data it works better as a way to spend them where they tell us most.

Why so many follow-ups. Most of the extra length comes from one chain of events. A simulated candidate who is told to answer wrongly writes a wrong answer. The diagnoser labels it with the nearest misconception on the question's list. That earns a follow-up, and a second matching

reply confirms the misconception. Looking at the transcripts, these extra confirmations are of two kinds:

- *The candidate really did state the belief.* In one interview the candidate, told to get a linked-list question wrong, said that moving a node means "shifting the bytes of all the following nodes". That is exactly the listed misconception. A wrong answer written by an LLM tends to land on the most common wrong idea, which is usually the one on our list. Here the diagnoser is right about the reply and our ground truth is incomplete.
- *The diagnoser stretched.* In another, a candidate said removing the top of a heap takes $O(n)$ time. The listed misconception is believing it takes $O(1)$. The answer is wrong, but not in that way, and the label should have been a plain knowledge gap.

The measured precision of 18% mixes these two, so it understates the diagnoser. We do not yet know the split. Two teammates will label a sample of these replies by hand before the final report. The second kind can be fixed in the diagnosis prompt, by requiring the reply to state the listed belief before that misconception can be chosen.

Premature stops. 38% of A2 interviews stopped as "confident" while at least one of the three skill levels was wrong. This is a strict measure (one wrong skill out of three is enough) and it follows from the per-skill accuracy, but it does mean that a confident report is not always right, and a human should treat the levels as estimates.

8 Limitations

- **Simulated candidates.** Real candidates are less consistent and less tidy. The results show how the interviewer behaves against a controlled population, not how it would rate real people. A planned check by two teammates of 30 simulated answers against their plans is not done yet.
- **Small sample.** 24 candidates give wide intervals. The accuracy gain is clear; the recall target is not settled either way.
- **One grader model.** Every grade comes from one Gemini model. We have not yet compared it with hand marks.
- **Bank not fully reviewed.** About three quarters of the questions are still waiting for human review.
- **Tuning on stubs.** The cut points and the threshold were tuned with a perfect grader. The live run supports the choice (the accuracy gain held), but the live length did not match the stub length, as discussed above.

9 Next steps

1. Finish the question review and fix the errors it finds.
2. Hand-label the extra misconception confirmations, then tighten the diagnosis prompt so a misconception is only chosen when the reply states it. We expect this to cut both the follow-ups and the length.
3. Run all 60 candidates on B1, A2, A4 (skill-only scorecard) and A5 (job-blind retrieval). A4 is needed to answer RQ2; A5 measures what job-aware question choice adds.
4. Check the grader against hand-marked answers, and the simulated answers against their plans.
5. Look at the fit rule's 80% threshold, which sends about half the candidates to review.

Reproducing this report

Every figure and every number in this report is produced by `report/make_figures.py` from the saved interview results, so nothing is copied by hand. The interviews themselves come from `scripts/run_experiment.py` and the summary table from `scripts/evaluate.py`.

References

[1] E. Al-Hossami and R. Bunescu. McMining: automated discovery of misconceptions in student code. In *EACL* (short papers), 2026. arXiv:2510.08827.

[2] R. D. Bock and R. J. Mislevy. Adaptive EAP estimation of ability in a microcomputer environment. *Applied Psychological Measurement*, 6(4):431–444, 1982.

[3] L. Chiodini, I. Moreno Santos, A. Gallidabino, A. Tafliovich, A. L. Santos and M. Hauswirth. A curated inventory of programming language misconceptions. In *ITiCSE*, pages 380–386, 2021.

[4] K. Karpierz and S. A. Wolfman. Misconceptions and concept inventory questions for binary search trees and hash tables. In *SIGCSE*, pages 109–114, 2014.

[5] G. N. Masters. A Rasch model for partial credit scoring. *Psychometrika*, 47(2):149–174, 1982.

[6] D. Miedema, E. Aivaloglou and G. Fletcher. Identifying SQL misconceptions of novices: findings from a think-aloud study. In *ICER*, pages 355–367, 2021.

[7] P. Oli, R. Banjade, A. M. Olney and V. Rus. Can LLMs identify gaps and misconceptions in students' code explanations? arXiv:2501.10365, 2025.

[8] J. Yu, Y. Zhuang, Y. Sun, W. Gao, Q. Liu, M. Cheng, Z. Huang and E. Chen. TestAgent: an adaptive and intelligent expert for human assessment. In *Findings of ACL*, pages 724–747, 2025.

[9] Y. Zhuang, Q. Liu et al. Survey of computerized adaptive testing: a machine learning perspective. arXiv:2404.00712, 2024. Accepted to *IEEE TPAMI*.